

Erklärt Netznähe die Latenzunterschiede?

Eine drei-schichtige Messstudie zu Cloud-AI-Sprachdiensten (STT · LLM · TTS) aus EU-Sicht

Konsultation · Setup-Stand

Agenda

1

Setup

SCHWERPUNKT

Vantage Point · Provider-Matrix · Messdesign

2

Methodik

3 Schichten · Edge/Host · Eichung · Kampagne

3

Analyse

Kernbefund · Interpretation · Limitationen (Vollkampagne)

4

Abschluss

Stand · nächste Schritte · Diskussion

Forschungsfrage

- Pipeline: **STT** → **LLM** → **TTS** (sequenziell)
- Latenz entscheidet die Nutzbarkeit
- **Kernfrage:** Netz/Infrastruktur vs. Backend?
- Faktoren: **RTT, Protokoll, Hosting-Region**
- Bewusst offen — **auch ein Nein zählt**
- Zweite Achse: **Verfügbarkeit** (separat)



ABSCHNITT 1

Setup

- Vantage Point
- Provider-Matrix
- Messdesign



Vantage Point

- **AWS EC2 eu-central-1** (Frankfurt)
- **c6i.large** — bewusst NICHT burstable
- Burstable würde drosseln → **Latenz-Sprünge ohne Netz-Ursache**
- **CPU-Steal-Time** je Slot geloggt
- **IPv4** durchgängig erzwungen
- **OpenSSL 3.x** (LibreSSL meldete TLS-Zeiten falsch)
- Limitation: **ein Vantage Point** — Kernbefund unberührt

Provider-Matrix · 9 Endpunkte

- 9 Endpunkte über 6 Anbieter × 3 Kategorien
- Feste Inputs je Kategorie
- Fairer Cross-Provider-Vergleich

Kat.	Anbieter	Modell	Region (deklariert)	Protokoll
STT	Deepgram	Nova-3	USA (Multi-DC, Round-Robin)	WebSocket
STT	Rev.ai	English	USA	WebSocket
STT	Azure	Standard Neural	Italien (Italy North)	WebSocket
LLM	OpenAI	gpt-4o-mini	USA (GPU)	HTTPS + SSE
LLM	Groq	llama-3.1-8b-instant	USA (LPU)	HTTPS + SSE
LLM	Mistral	mistral-small-2603	EU / Frankreich	HTTPS + SSE
TTS	Deepgram	Aura-2	USA (Multi-DC, Round-Robin)	HTTPS Streaming
TTS	OpenAI	tts-1	USA	HTTPS Streaming
TTS	Azure	Standard Neural	Italien (Italy North)	HTTPS Streaming

Messdesign · Cold-Start · interleaved

- **Cold-Start:** frische TCP+TLS je Call
- Kein Pooling · kein Keep-Alive
- **Feste Inputs** je Kategorie
- STT 5 s-WAV · LLM Capital-Prompt · TTS Gruß
- **Interleaved** Round-Robin, 100 Runden
- Startreihenfolge rotiert
- Fehlschlag blockiert die Runde nicht

ABSCHNITT 2

Methodik

- 3 Schichten · RTT · Edge/Host
- Paket-Eichung · Metrik-Uhr
- Kampagne & Aggregation

2

Drei-Schichten-Architektur

- Die Frage liefert die **Zerlegung**
- **Layer 1** → **Host** · **Layer 3** → **volle URL**
- Differenz = **Backend (Hardware + Modell)**
- **Layer 2** macht die Leitung prüfbar

LAYER 3 · API-Latenz (Cold-Start)

DNS / TCP / TLS / WS-Handshake + ttfp · ttft · ttfa + total

misst über die volle URL

LAYER 2 · Paketebene (PCAP)

SYN → SYN-ACK · Round-Trips · Inter-Arrival · Retransmits

leicht den Connect-Timer

LAYER 1 · Infrastruktur

DNS · RTT/Ping (TCP primär, ICMP-Check) · TLS · Traceroute · ASN

misst die Netznähe zum Host

Layer 1 · RTT via TCP-Handshake

- **TCP-Ping** (SYN→SYN-ACK :443) = primär
- Vergleichbar bei **allen 9** (ICMP teils geblockt)
- Trifft den **echten API-Port**
- Deckt sich mit dem **Connect-Timer aus Layer 3**
- Validierung: **TCP-Ping $\approx$ ICMP-Ping**

Layer 1 · Edge oder Host?

Offene Frage: Drei Endpunkte antworten aus Frankfurt mit ~1 ms RTT — woher?

- **Erklärung:** Cloudflare-Edge FRA, nicht Backend
- ~1 ms = reale Edge-Eigenschaft, kein Fehler
- **Edge nur, wenn alle drei zutreffen:**
 - (a) **einstellige ms** aus FRA
 - (b) Ziel-IP in **AS13335** (CDN)
 - (c) Traceroute endet **im CDN-AS**

Layer 2 · Paket-Eichung des Connect-Timers

Validierung: App-Timer unabhängig gegen die Paketebene (PCAP) geeicht.

- **30 Cold-Starts je Provider** (paketvalidiert: 30/30 bzw. 28/30)
- App-Timer vs. **SYN→SYN-ACK** (PCAP)
- Geeicht: **nur der Connect-Timer**
- ttft/ttfa: gleiche Uhr, aber **nicht direkt** paket-geeicht
- Layer 2 = **Konsistenz-Check**, kein 2. Beweis

Anbieter	RTT-Klasse	App-Timer	SYN→SYN-ACK	Δ (Median)
Azure (Italy North)	~11 ms	11,29 ms	11,18 ms	+0,11 ms
Deepgram (US)	~139 ms	139,01 ms	138,87 ms	+0,12 ms

→ Connect-Timer trifft die Latenz auf der Leitung auf ~0,1 ms genau (Median-Offset +0,11 ms; erster Messpunkt als Outlier entfernt).

Metrik-Asymmetrie · ab wann läuft die Uhr?

- **Klar deklariert:** ab wann die Uhr läuft
- Jeder Call ist Cold-Start: Connect **immer** gemessen
- STT **tftp**: nur Engine ab erstem Audio-Chunk (Connect **separat**)
- LLM/TTS **ttft/ttfa**: Connect **enthalten** (ab Request)
- E2E: jeder Anbieter-Connect **genau einmal**

Kategorie	Primärmetrik	Uhr startet bei	Connect
STT	tftp (endpointing-frei)	erstem Audio-Chunk	separat gemessen · 1×-Realtime
LLM	ttft	Absenden des Requests	enthalten (im ttft)
TTS	ttfa	Absenden des Requests	enthalten (im ttfa)

Kampagne & Aggregation

Kampagne

- 7 Tage × 8 UTC-Slots × n=100
- = 56 Slots = **50.400 Calls**
- UTC-verankert: US-Hoch + -Tief
- Erhoben & ausgewertet: **alle 56 Slots** (A4)

Aggregation & Inferenz

- Headline = **Median der Slot-Mediane**
- Jede Zahl: **Bootstrap-95%-CI**
- ‚schneller‘ getestet mit **Mann-Whitney / Bootstrap**
- **Verfügbarkeit** = eigene Achse

ABSCHNITT 3

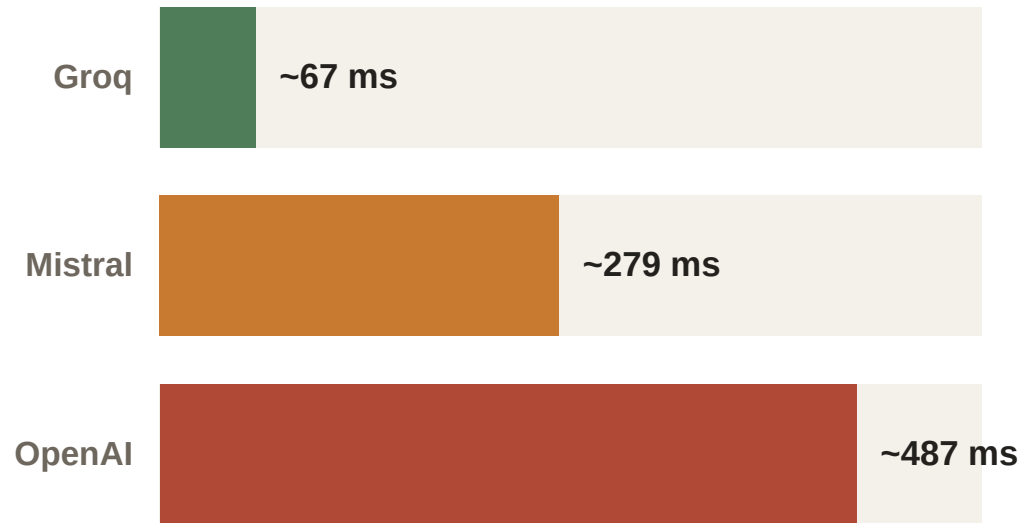
Analyse

Vollkampagne 56/56 · A4-Median

- Kernbefund
- Interpretation
- Limitationen

3

Kernbefund · gleicher Edge, ~7,3×



LLM-ttft · connect-inkl. · 56/56 Slots, A4-Median

- Alle 3 LLM @ **Cloudflare FRA**
- **100 % des Traffics** belegt (2 CF-IPs, AS13335, ~1 ms)
- ttft streut **~7,3×** (Groq→Mistral→OpenAI)
- **Pro IP stabil** (kein Edge-Effekt)
- **Region-Reihenfolge gedreht:** Mistral (EU) langsamer als Groq (US)
- Gleiches Netz, ~7,3× → **Netznähe erklärt das nicht**

Interpretation · der Kompass

Verteidigbar ist die negative Aussage: ‚Netznähe erklärt die Latenzspreizung nicht.‘

- Schritt zu Backend trägt **Modellgrößen-Confound**
- Groq = kleinstes Modell + **LPU**
- → **Backend (Hardware + Modell)** statt Geografie
- **2. Beleg (TTS):** OpenAI-TTS @ CF-FRA → ~942 ms ttfa = fast **reines Backend** (connect ~1 ms)



ABSCHNITT 4

Abschluss

- Stand
- Nächste Schritte
- Diskussion



Stand & nächste Schritte

- **Messinfrastruktur steht** und läuft stabil
- → offene Punkte betreffen nur **Doku/Reporting**, nicht die Daten
- Stop ~23.06. → Statistik (**Bootstrap-CI + Inferenz**)
- Zweitgutachten: **Prof. Färber zugesagt** (Exposé, Gespräch Juli)

Klassifikator & Per-IP-Belege

- Edge nur, wenn **alle drei** zutreffen
- (a) **einstellige ms** FRA :443
- (b) IP in **AS13335** (Team-Cymru)
- (c) Traceroute endet im CDN-AS
- Belegt über **alle getroffenen IPs**
- Deepgram: **zwei US-Carrier** (AS6461 / AS174), ~101–140 ms
- Rev.ai **AS16509** ~140 ms · Azure **AS8075** ~11 ms
- Bei Edge ist connect $\approx N \times \text{RTT}$ zwangsläufig → **kein eigener Beleg**

Aggregation & Inferenz (Detail)

- p50/p90 slot-aufgelöst · p95 nur gepoolt + CI · p99 nicht
- Faustregel: $n \cdot (1-q) \geq 5-10$
- **Stark rechtsschief** → Bootstrap statt t
- E2E: **Monte-Carlo-Faltung** der 3 Phasen
- Joint-Completion / **Pareto-Front** (Latenz vs. Zuverlässigkeit)
- IP-Quelle: **aufgelöste Connect-IP** — Fehlschläge nie über Null-Werte filtern

Fehlerbehandlung & Verfügbarkeit

- Erfolg = **inhaltlich gültiger Output** (nicht nur Verbindung)
- Timeout-Filter: Teilstring „**ReadTimeout**“ klassifizieren
- nicht nach `error=='timeout'` (verfehlt fast alle echten Timeouts)
- OpenAI-TTS **~3,1 % ReadTimeouts** (173/5600, Vollkampagne)
- → Latenz nur erfolgreiche Calls + Asterisk

Kategorie	Connect-TO	Response-TO	Mindest-Output für „Erfolg“
STT	10 s	30 s	nicht-leeres Final-Transkript (≥ 1 Wort)
LLM	10 s	30 s	≥ 1 Wort UND ≥ 3 SSE-Content-Chunks
TTS	10 s	30 s	Audio-Body ≥ 1.000 Bytes (dekodierbar)

Mess-Stack (Code)

- Pro Schicht ein eigenes Werkzeug: **CLI** für L1/L2, **Python-Timer** für die API-Latenz
- STT ohne SDK (**rohe WebSockets**) · IPv4 erzwungen · Cold-Start je Call

Layer	Werkzeug / Library	Kernfunktion / Metrik
L1 · RTT	Python socket	create_connection(ip,443) → SYN→SYN-ACK = 1 RTT (min+median)
L1 · DNS	dig @ 8.8.8.8 / 1.1.1.1 / 9.9.9.9	IPs je Resolver + TTL
L1 · ASN	Team-Cymru (dig TXT)	ASN je IP → Edge-Beleg (Bed. b)
L1 · TLS	Python ssl + openssl s_client	TLS-Version · Cipher · ALPN · Cert
L1 · Route	tracert -T -p 443 + Cymru	endet die Route im CDN-AS? (Bed. c)
L2	tcpdump → analyze.py	SYN→SYN-ACK vs. App-Timer (Eichung)
L3 · LLM/TTS	httpx (HTTPS + SSE / Stream)	ttft/tfa ab Request · IPv4 · effective_model aus SSE
L3 · STT	websockets + asyncio.gather	ttfp ab erstem Audio-Chunk · 1×-realtime gepaced
Runner	run.py	interleaved Round-Robin · flock · Slot-Deadline → JSONL

Endpunkte & Auflösung

■ 9 Endpunkte über 6 Anbieter — live per DNS aufgelöst, per ASN + RTT klassifiziert

Kat. · Anbieter	Host	Protokoll	Auflösung (ASN · RTT)	Term.
STT Deepgram	api.deepgram.com	WSS	AS6461/AS174 US · ~101–148 ms	Host
STT Rev.ai	api.rev.ai	WSS	AS16509 AWS-US · ~140 ms	Host
STT Azure	italynorth.stt.speech.microsoft.com	WSS	AS8075 MS-Italy · ~11 ms	Host
LLM OpenAI	api.openai.com	HTTPS+SSE	AS13335 Cloudflare · ~1 ms	Edge
LLM Groq	api.groq.com	HTTPS+SSE	AS13335 Cloudflare · ~1 ms	Edge
LLM Mistral	api.mistral.ai	HTTPS+SSE	AS13335 Cloudflare · ~1 ms	Edge
TTS Deepgram	api.deepgram.com	HTTPS-Stream	AS6461/AS174 US · ~140 ms	Host
TTS OpenAI	api.openai.com	HTTPS-Stream	AS13335 Cloudflare · ~1 ms	Edge
TTS Azure	italynorth.tts.speech.microsoft.com	HTTPS-Stream	AS8075 MS-Italy · ~11 ms	Host

Anatomie einer Latenzzahl

- **ttft** = ab Request-Absenden bis erstes Token — connect-**inklusive**
- darin: DNS + TCP + TLS = **connect- Σ ~7 ms** (für alle 3 ~gleich)
- davon nur **~1 ms reine TCP-RTT** (= Netznähe, Layer 1) · TLS ~5 ms
- Rest = **Backend** ~60 → ~479 ms → trägt die **~7,3x-Spreizung**

LLM	DNS	TCP (=RTT)	TLS	connect Σ	ttft	→ Backend
Groq	0,3	1,3	5,7	~7,3	~67	~60
Mistral	0,3	1,1	5,8	~7,2	~279	~272
OpenAI	1,4	1,2	4,9	~7,5	~487	~479